

If the number is large, the screen scrolls. If the terminal does not support scroll bars (as was the case for many years), there is no way to look at the entries that have scrolled past. To help overcome this, we use commands like ‘more’ and ‘less’. These allow us to view the output on a page-by-page basis. The command typically used is:

```
ls -l | less
```

What is the shell doing here? What looks like a single command is actually two commands executing one after the other, *ls* and *less*. The pipe (‘|’) connects the two programs, but the connection is managed by the shell. Because of the pipe character, the shell connects the two programs – it connects the standard output of the *ls* command and connects it to the standard input or standard in or stdin of *less*. The pipe feature allows us to take the output of any program and provide it as the input to another program – without us having to do any changes to the programs. This is the philosophy of many UNIX/Linux applications — keep the applications simple and then combine many applications together to achieve the end result, rather than having one program do many things.

If needed, we can redirect the output of *ls* to a file and then view it using ‘vi’. For this, we use the command:

```
ls -l > /tmp/my_file.txt
vi /tmp/my_file.txt
```

In this case, the output of *ls* is being redirected to a file. This is managed by the shell, which understands the ‘>’ symbol to mean redirection. It treats the token that follows as a file.

Automation using shell

This ability to combine commands is one of the key elements for the creation of automation scripts using shell commands. In my most recent project, we were executing Python/Spark (PySpark) applications using cluster mode. Each application executed many structured query language (SQL) statements – SparkSQL. To keep track of application progress, we were printing details about the SQL being executed. This allowed us to maintain a log of what was happening in the application. As the applications were executed in cluster mode, to view the log, we had to use the *yarn* command as follows:

```
yarn log -applicationId [application_id]
```

In most cases, the log produced by an application was very large. So we typically piped the log to ‘less’ or redirected it to a file. The command we used was:

```
yarn log -applicationId [application_id] | less
```

Our development team had a strength of 40 people. Each one had to remember this command. To make it simpler, I converted this command into a bash script. For this, I created a file with a ‘.sh’ extension. On UNIX and Linux systems, file extension does not matter. As long as the file is an executable, it will work. Extensions have significance on MS Windows.

Important thing to remember

The shell is an interpreter. This means that it will read the program line by line and execute it. The limitation of this approach is that errors (if any) are not identified upfront. Errors are not identified till they are read and executed by the interpreter. In short, we can have a shell program that will execute perfectly for the first 20 lines and then fail due to a syntax error on line 21. When the script fails at line 21, the shell does not unroll/undo the previous steps. When such a thing occurs, we have to correct the script and start execution from the first line. Thus, as an example, if we have deleted a few files before encountering an error, execution of the shell script will stop, but the files are gone forever.

The script I created was:

```
#!/bin/bash
yarn log -applicationId 123 | less
```

...where 123 was the application ID.

The first two characters of the first line are magic characters. They tell the script that this is an executable file and the line contains the name of the program to be used for execution. The remaining lines of the script are passed to the program mentioned. In this case, we are going to execute bash. Even after including the first line, we have to apply execute permissions to the file using:

```
chmod +x my_file.sh
```

After giving execute permissions to the file, we can execute it as:

```
./my_file.sh
```

If we do not give execute permissions to the file, we can execute the script as:

```
sh ./my_file.sh
```

Passing parameters

You will realise quickly that such a script is handy, but becomes useless immediately. Each time we execute the Python/Spark application, a new ID is generated. Hence, for each run, we have to edit the file and add the new application ID. This